

ALGORITMOS E PENSAMENTO COMPUTACIONAL



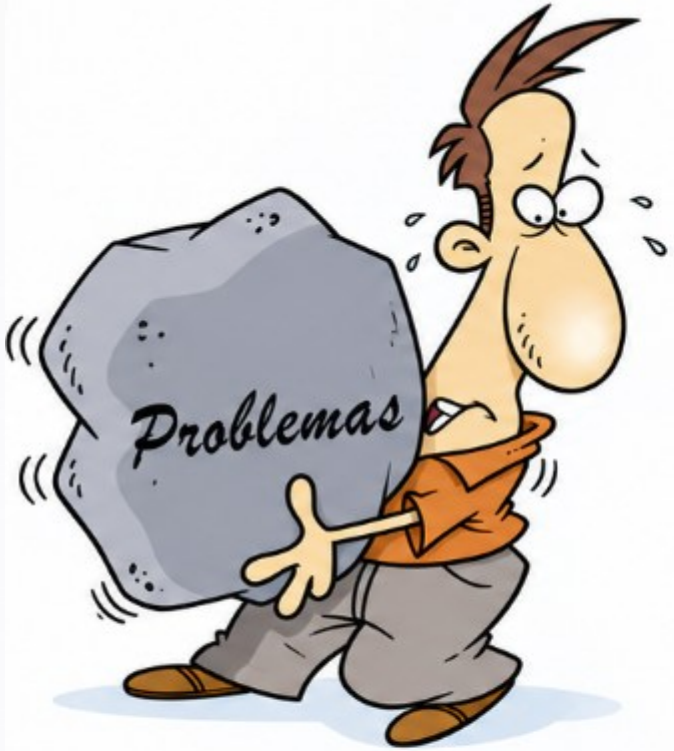
Manipulação de Arquivos

- Leitura e escrita em arquivos texto
- Abertura, fechamento e modos de acesso
- Processamento de dados armazenados

Professor Marco Antonio

2º semestre 2026

Reflexão inicial




```c
#include <stdio.h>

int main() {
    float notas[4] = {7.5, 8.0, 6.5, 9.0};
    printf("Notas da turma:\n");
    for (int i = 0; i < 4; i++) {
        printf("%.1f ", notas[i]);
    }
    return 0;
}
```

```
Notas da turma:
7.5 8.0 6.5 9.0
```

7.5
8.0
6.5
9.0

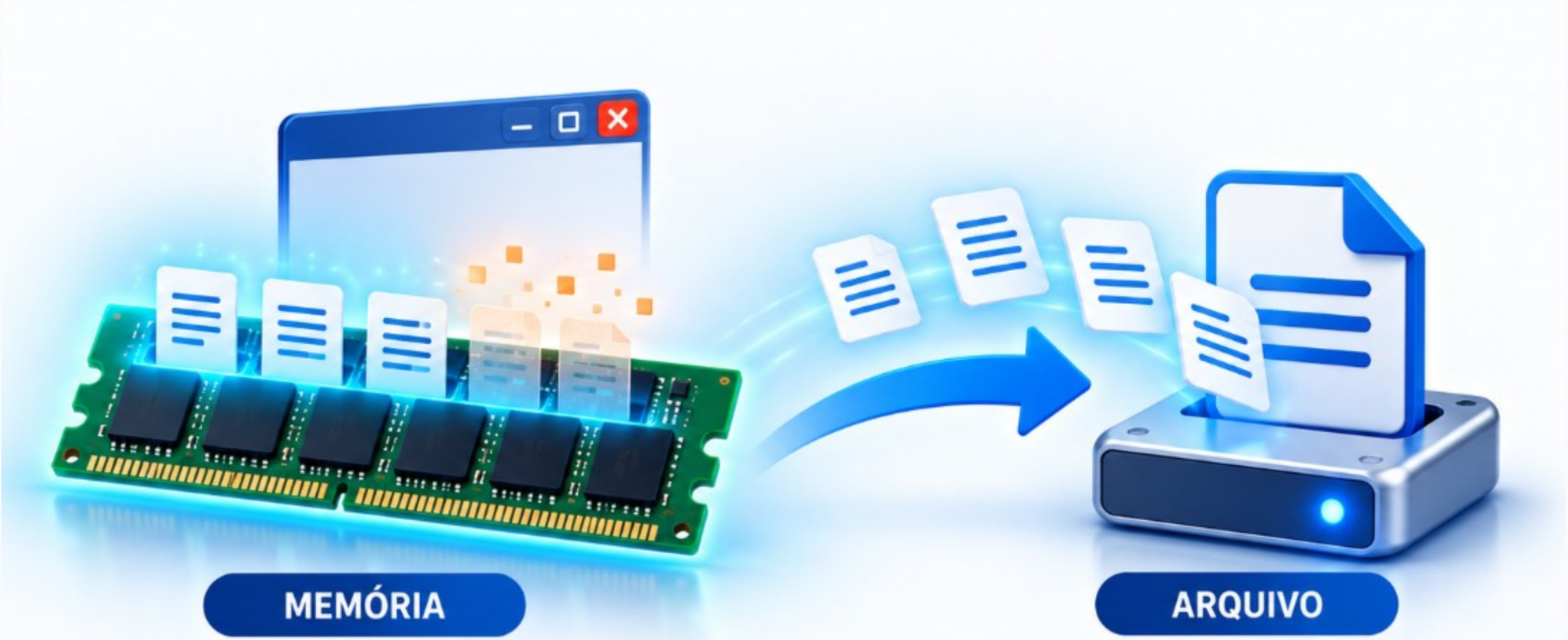
(vazio)

Um programa registra as notas de uma turma em um vetor. Ao encerrar a execução, os dados desapareceram.

Para refletir

- Por que os dados foram perdidos?
- Como recuperar as notas em outra execução?
- Onde os dados poderiam ser armazenados?
- O que muda com centenas de registros?

Conceitos abordados nesta aula



Variáveis têm valor e endereço

Uma variável pode ser observada por duas informações:

valor	**20**	dado armazenado
endereço	**&idade**	localização da variável na memória

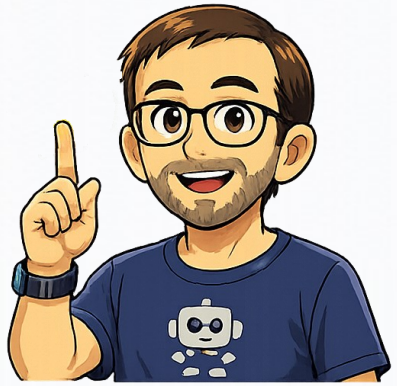
Conceitualmente, considere uma variável que armazena a idade de uma pessoa:

Code - fluxo.c

```
1 int idade = 20;
```



O operador **&** permite obter o endereço de uma variável.



O que é um ponteiro?

Um **ponteiro** é uma variável que armazena o endereço de outra variável.

CÓDIGO

```
int idade = 20;
int *ponteiro = &idade;
```



SIGNIFICADO DAS EXPRESSÕES

`idade`	valor armazenado em `idade`
`&idade`	endereço de `idade`
`ponteiro`	endereço armazenado no ponteiro
`*ponteiro`	valor encontrado no endereço apontado

ALTERAÇÃO INDIRETA

```
*ponteiro = 25;
```

RESULTADO EM `idade`

20 → 25

O ponteiro não guarda diretamente o dado: ele indica onde o dado pode ser encontrado.

Exemplo

Code - ponteiro.c

```c
#include <stdio.h>

int main(void)
{
    int idade = 20;
    int *p;

    p = &idade;

    printf("Valor de idade: %d\n", idade);
    printf("Endereço de idade: %p\n", &idade);
    printf("Valor guardado em p: %p\n", p);
    printf("Valor apontado por p: %d\n", *p);

    return 0;
}
```

A variável **p** guarda o endereço da variável idade

```
┌──(Prof. Marco 💀 PICHAU)-[output]
└─$  & .\'ponteiro.exe'
Valor de idade: 20
Endereço de idade: 00000068797FF874
Valor guardado em p: 00000068797FF874
Valor apontado por p: 20
```

Por que utilizar um ponteiro?

Os ponteiros permitem trabalhar com endereços de memória e acessar dados indiretamente. **Em C, eles são importantes, por exemplo, para:**

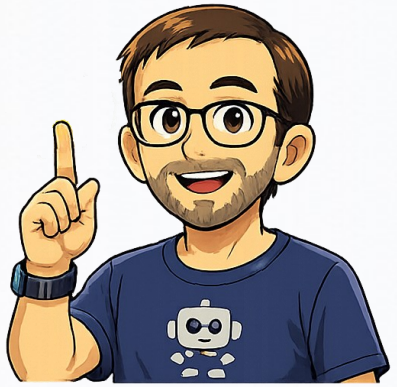
- modificar variáveis por meio de funções;
- evitar cópias desnecessárias de grandes estruturas;
- percorrer vetores e **`Strings`**;
- acessar realizar alocação dinâmica de memória.

Ao manipular arquivos, o programa precisa manter informações sobre um fluxo aberto.

O ponteiro **`arquivo`** permite ao programa acessar a estrutura associada a esse fluxo.

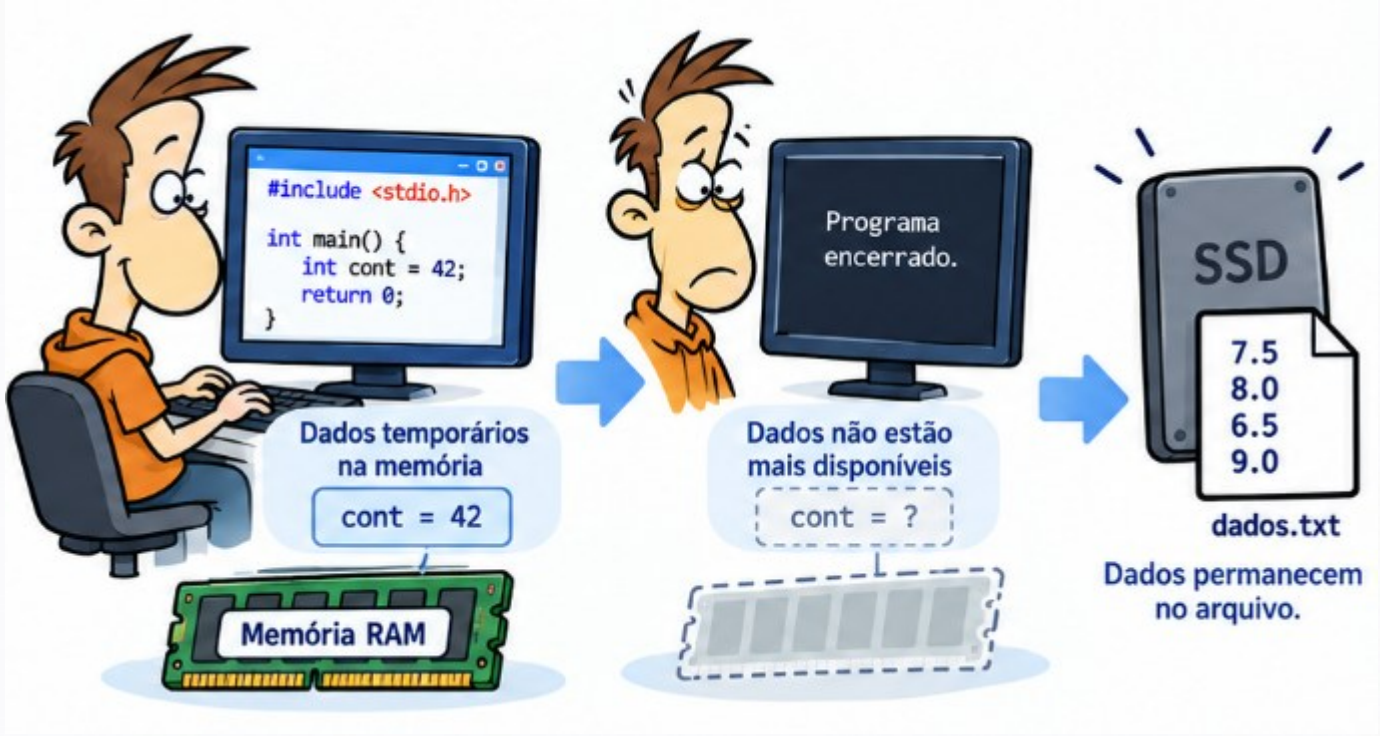
Essa estrutura mantém informações como:

- modo de abertura;
- posição atual de leitura ou escrita;
- estado do fluxo;
- buffer utilizado nas operações.



`FILE *arquivo` funciona como uma referência ao fluxo usado para ler ou escrever dados no arquivo, como veremos a seguir.

Persistência de dados



As variáveis armazenam dados na memória durante a execução do programa.

Quando o programa termina, esses dados deixam de estar disponíveis.

Um arquivo permite:

- guardar dados em um dispositivo de armazenamento;
- recuperar informações em outra execução;
- compartilhar dados com outros programas;
- processar registros sem digitá-los novamente.

PERSISTÊNCIA | Permanência dos dados além da execução do programa que os produziu.

Arquivos de texto x arquivos binários

Arquivo texto



- Armazena dados como caracteres
- Pode ser aberto em um editor de texto
- Exemplos: .txt, .csv, .json
- Favorece a inspeção manual

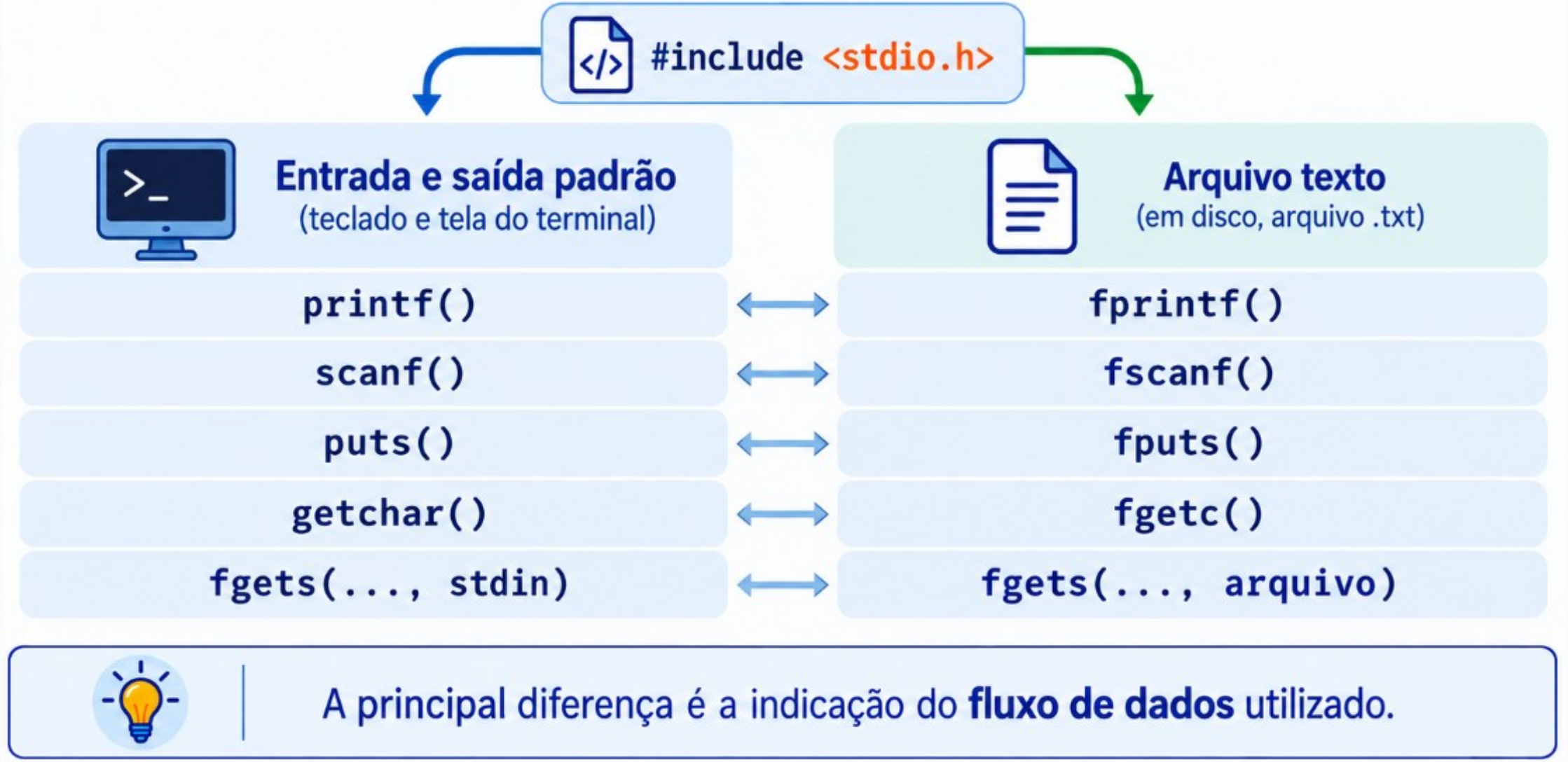
Arquivo binário



- 0101 1010 — Armazena dados como bytes
- Depende de um programa para interpretação
- Exemplos: .jpg, .mp3, .exe
- Pode ser mais compacto e eficiente

DELIMITAÇÃO | Nesta aula, trabalharemos somente com **arquivos texto.**

Da tela para o arquivo



Fluxo básico de manipulação de um arquivo em C

Declarar um ponteiro do tipo **FILE**.

Abrir o arquivo com **fopen()**.

Aula 08 - Manipulação de Arquivos - fluxo.c

```c
FILE *arquivo = fopen("dados.txt", "r");

if (arquivo == NULL) {
    printf("Não foi possível abrir o arquivo.\n");
    return 1;
}

/* Leitura ou escrita */

fclose(arquivo);
```

Verificar se a abertura funcionou.

Ler ou escrever os dados.

Fechar o arquivo com **fclopen()**.

Fluxo básico de manipulação de um arquivo em C

```
FILE *arquivo;
arquivo = fopen("dados.txt", "r");
```



FILE é um tipo definido em **stdio.h**.

O ponteiro mantém:

- modo de abertura
- posição atual de leitura ou escrita
- estado do fluxo
- buffer usado nas operações

fopen() retorna **NULL** quando não consegue abrir o arquivo.

Modos de acesso essenciais

Modo	Operação	Se não existir	Se existir
"r"	leitura	a abertura falha	preserva o conteúdo
"w"	escrita	cria o arquivo	apaga o conteúdo anterior
"a"	acréscimo	cria o arquivo	escreve no final

ATENÇÃO
Abrir um arquivo existente com "w" elimina seu conteúdo anterior.

```
fclose(arquivo);
```

O fechamento conclui operações pendentes e libera os recursos usados pelo sistema.

```
fopen("dados.txt", "w");
```

Caminho relativo: o arquivo é procurado ou criado no diretório de trabalho do programa.



Ampliação posterior: **r+**, **w+** e **a+**.

Vamos praticar





Exercícios de aplicação

Um estudante quer gerar uma tabuada e consultá-la depois, mesmo após encerrar o programa.

1- Desenvolva um algoritmo e um programa em C que leia um número inteiro e grave em um arquivo chamado `tabuada.txt` as multiplicações desse número de 0 a 10.

Caso de teste

```
Entrada:
Digite um número inteiro: 7

Saída:
7 * 0 = 0
7 * 1 = 7
7 * 2 = 14
[...]
7 * 9 = 63
T * 10 = 70
```

Planejamento do Exercício 1

ENTRADA

Número da tabuada

7

Exemplo: o usuário informa o número da tabuada.

PROCESSAMENTO

1. Abrir `tabuada.txt` no modo `"w"`
2. Repetir os valores de 0 a 10

`i = 0 ... 10`

3. Calcular e gravar cada multiplicação

```
resultado = numero * i;
gravar no arquivo;
```

4. Fechar o arquivo

tabuada.txt

SAÍDA

Arquivo texto com onze linhas

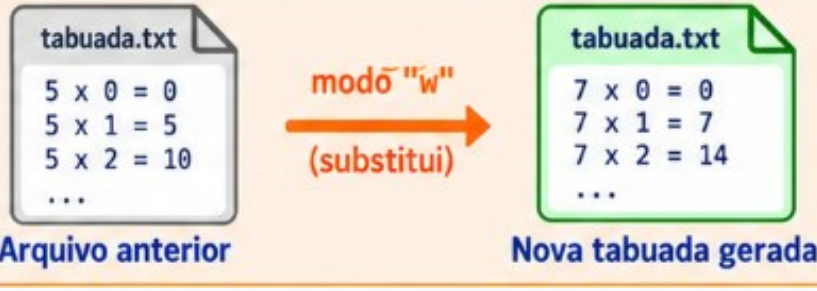
tabuada.txt

```
7 x 0 = 0
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

11 linhas

DECISÃO

O modo `"w"` é adequado porque cada execução deve gerar uma nova tabuada.



Exercícios de aplicação

Um aplicativo mantém o histórico do consumo semanal de água de uma residência. Como o arquivo pode conter registros anteriores, os novos dados devem ser acrescentados sem apagar as informações já armazenadas.

2- Desenvolva um algoritmo e um programa em C que Leia a quantidade de registros que serão cadastrados. Para cada registro, leia o dia da semana e o consumo de água em litros. Acrescente os dados ao arquivo `consumo.txt`, utilizando uma linha para cada registro.

Caso de teste

```
Entrada:
Dia: seg
Consumo: 3.5
Dia: ter
Consumo: 7.8
Dia: qua
Consumo: 9.25

Saída:
seg 3.50
ter 7.80
qua 9.25
```

Funções de leitura

Como ler o arquivo?

Podemos ler:

um caractere com	uma linha com	dados formatados com
`fgetc()`	`fgets()`	`fscanf()`

Organização dos dados	Função sugerida
texto livre, linha por linha	`fgets()`
campos com formato conhecido	`fscanf()`
análise individual de caracteres	`fgetc()`

A função escolhida deve ser **compatível** com o formato do arquivo e com o **processamento desejado**.

Posição atual de leitura

Cada operação de leitura avança a posição atual do arquivo.



1. `fscanf(arquivo, ...);` — Lê dados do registro 1 (e avança a posição).
2. `fgets(linha, sizeof(linha), arquivo);` — Continua do ponto em que a leitura parou (**registro 2**).

Após **`fscanf()`**, **`fgets()`** continua do ponto em que a leitura parou.

`rewind(arquivo);` | Retorna ao início

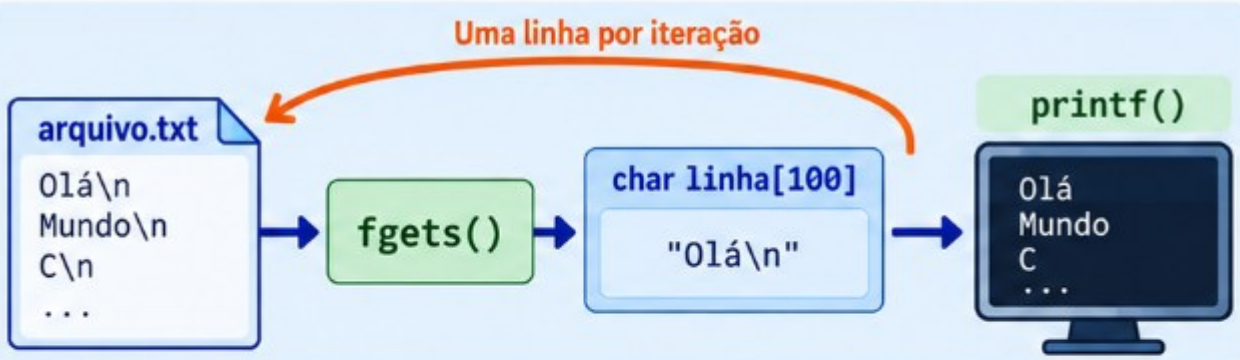


Outra possibilidade: fechar e abrir o arquivo novamente.

Leitura linha por linha com fgets()

```c
char linha[100];

while (fgets(linha, sizeof(linha), arquivo) != NULL) {
    printf("%s", linha);
}
```

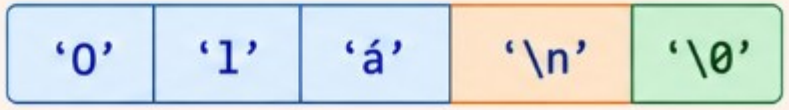


fgets():

- **\n** — Lê até encontrar uma quebra de linha
- Limita a quantidade de caracteres lidos
- **\0** — Adiciona **\0** ao final da string
- **NULL** — Retorna **NULL** quando não realiza outra leitura

ATENÇÃO

A quebra de linha lida pode permanecer **dentro da string.**



Exercícios de aplicação



Um estudante quer gerar uma tabuada e consultá-la depois, mesmo após encerrar o programa.

3- Desenvolva um algoritmo e um programa em C que abra o arquivo `tabuada.txt`, leia todas as linhas e exiba seu conteúdo na tela.

Caso de teste

```
Arquivo de Entrada:
7 x 0 = 0
7 x 1 = 7
7 x 2 = 14

Saída na tela:
7 x 0 = 0
7 x 1 = 7
7 x 2 = 14
```

Leitura formatada com fscanf()

notas.txt

```
1  25001234  8.5
2  25005678  6.0
3  25009876  4.5
```

`char rgm[12];`

`float nota;`

Leitura de um registro (uma linha)

```
fscanf(arquivo, "%11s %f", rgm, &nota);
```

Vetor de caracteres para o RGM

`char rgm[12];`

0	1	2	3	4	5	6	7	8	9	10	\0

11 caracteres disponíveis

%11s reserva uma posição do vetor para o caractere **\0**.

Princípio de compatibilidade

O formato do comando deve corresponder à organização dos dados no arquivo.

Condição correta de leitura

```
while (fscanf(arquivo, "%11s %f", rgm, &nota) == 2) {
    printf("%s: %.1f\n", rgm, nota);
}
```

O valor de retorno de fscanf()

`fscanf()` → `2` → ✔ RGM → ✔ nota → corpo do laço (executa)

2 campos lidos → **continua**

`fscanf()` → `≠ 2` → ✘ RGM → ✘ nota → encerra o laço (não executa)

retorno diferente de 2 → encerra

fscanf() retorna a quantidade de campos lidos e atribuídos corretamente.

Processamento de dados armazenados



O padrão de raciocínio permanece:
ENTRADA → PROCESSAMENTO → SAÍDA

Exercícios de aplicação



A coordenação do curso de letras recebeu um arquivo com o RGM e a nota final dos estudantes.

4- Desenvolva um algoritmo e um programa em C que abra o arquivo `notas.txt` e determine:

- quantidade de estudantes;
- média da turma;
- maior nota;
- quantidade de alunos aprovados (nota >= a 6,0).

Caso de teste

```
Arquivo de Entrada:
2500123-4 8.5
2500567-8 6.0
2500987-6 4.5

Saída na tela:
Estudantes: 3
Média da turma: 6.33
Maior nota: 8.50
Quantidade de alunos aprovados: 2
```

Decomposição do Exercício 4

1 ANTES DA REPETIÇÃO

Abrir o arquivo de entrada

- Abrir o arquivo
- Inicializar contador, soma e contadores auxiliares

```
contador  = 0
soma      = 0
aprovados = 0
```

2 DURANTE CADA LEITURA VÁLIDA

```
RGM: 25001234
Nota: 8.5
...
```

Para cada leitura válida

```
RGM: 25001234
Nota: 8.5
...
```

- Contar o estudante
- Acumular a nota
- Comparar com a maior nota
- Verificar se a nota alcançou 6,0

3 DEPOIS DA REPETIÇÃO

Se registros > 0

```
média = soma / contador
...
```

Relatório de Notas

Fechar arquivo

- Verificar se existe ao menos um registro
- Calcular a média
- Exibir o relatório
- Fechar o arquivo

PREPARAR (fora da repetição) → **REPETIR** (para cada leitura válida) → **CONSOLIDAR** (após a repetição)

Hora de praticar!!





Exercícios de aplicação



Uma loja registra em `vendas.txt` o código e o valor de cada venda concluída.

5- Desenvolva um algoritmo e um programa em C que leia o arquivo e exiba a quantidade de vendas, o total vendido e o maior valor registrado.



Para este exercício, você poderá criar o arquivo de entrada no bloco de notas ou criar um programa para armazenar os dados.

Caso de teste

Arquivo de Entrada:

```
101 150.00
102 89.90
103 320.50
```

Saída na tela:

```
Quantidade de vendas: 3
Total vendido: R$ 560.40
Maior venda: R$ 320.50
```

Juntando tudo: Consumo de energia



Uma residência utiliza deseja manter o histórico mensal do consumo de energia elétrica. Para tanto, você deve criar uma aplicação que salve em um arquivo (energia.txt), cada registro contendo o número do mês e o consumo correspondente em kWh.

6- Desenvolva um algoritmo e um programa em C que apresente o seguinte menu:

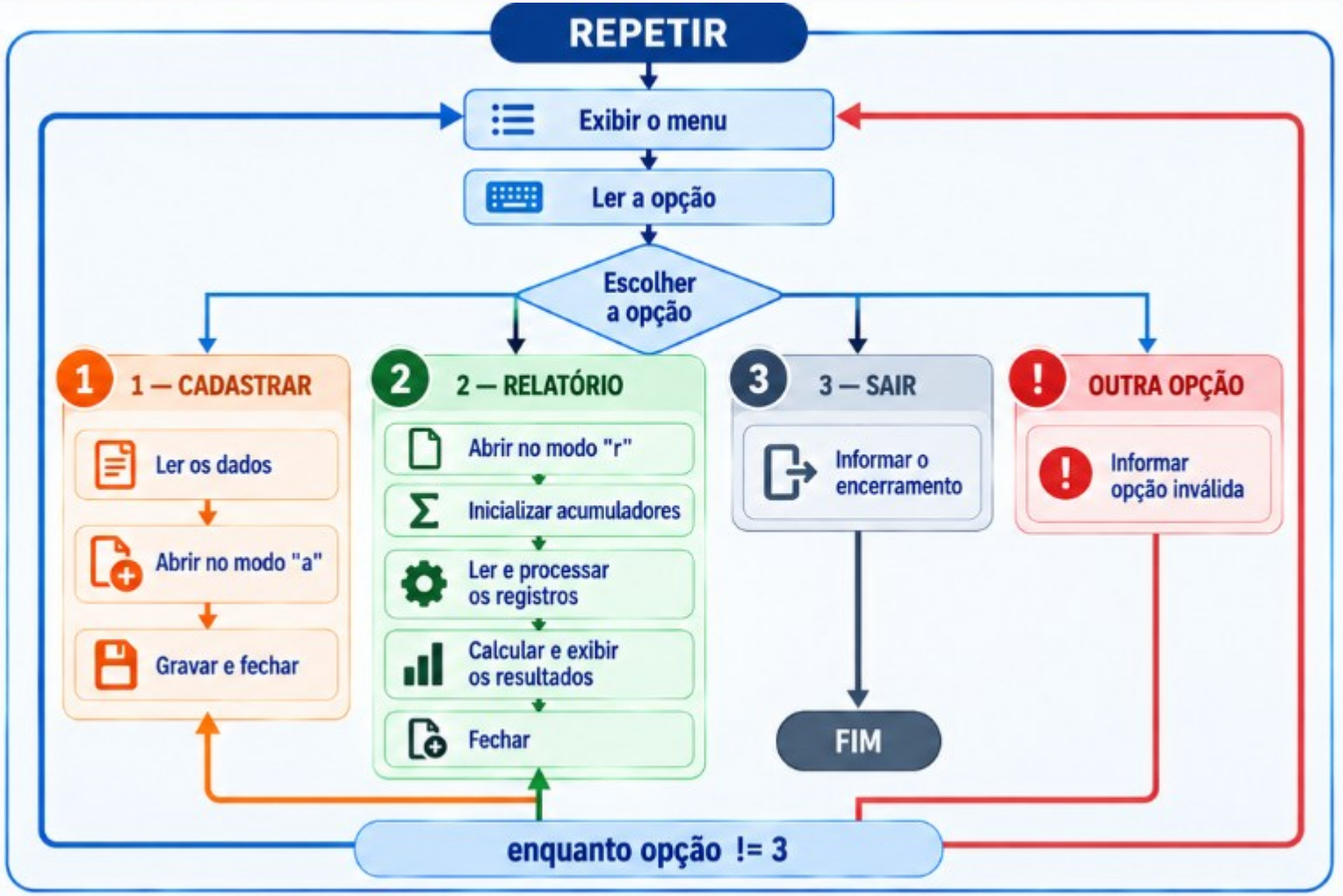
1- Cadastrar consumo
2- Exibir relatório
3- Sair

O programa deverá repetir o menu até que a opção 3 seja escolhida.

Planejamento da solução

Para o planejamento da solução, considere:

Opção	Modo	Processamento
Cadastrar	"a"	acrescentar um registro
Relatório	"r"	ler e processar todos os registros
Sair	—	encerrar a repetição



Resumindo tudo



Alguma dúvida????



Universidade Cruzeiro do Sul

CRÉDITOS

Material elaborado por:
Prof. Marco Antonio Sanches

Material base elaborado por:
- Prof. Alcides T. B. Júnior

Última atualização: 2026/2

CONTATOS


marco.sanches@cruzeirodosul.edu.br


marco-antonio-sanches-anastácio-601b9123